Ejercicio 10 — Par más cercano en una dimensión

Dificultad: ★★ **Conferencias:** C2 (El oficio algorítmico), C5 (Cotas mínimas)

Enunciado

> *Par más cercano en una dimensión. Demuestra $\Omega(n \log n)$ en árboles de decisión algebraicos por reducción desde distinción de elementos. Después di qué algoritmo de la conferencia 2 rompe la cota, en qué modelo, y adáptalo a una dimensión.*

C2 §3 ya resolvió el *algoritmo* del par más cercano en 1D (ordenar y mirar consecutivos, $O(n \log n)$). C2 §8 demostró la cota en el plano con el mapa $x_i \mapsto (x_i, 0)$, y C2 §9 dio el algoritmo aleatorio que la rompe. Este ejercicio hace tres cosas: (1) rehace la cota en 1D, donde la reducción desaparece y se puede dar además una demostración que no cita a Ben-Or; (2) identifica el modelo que escapa a la cota; (3) adapta la rejilla de C2 §9 a una dimensión, **rehaciendo** la cuenta geométrica en vez de copiar las constantes del plano. Lo "verificado" está en `verif.py`.

1 La cota: $\Omega(n \log n)$

El modelo

Un árbol de decisión algebraico (C2 §8) prueba en cada nodo el signo de un polinomio de la entrada y ramifica en tres ($<$, $=$, $>$); las hojas son respuestas, y el costo es la profundidad. Hay un detalle que importa: **las hojas contienen respuestas discretas**. Un árbol para el par más cercano no devuelve el número δ, devuelve en cada hoja *qué par* (i, j) es el más cercano. La figura 1 muestra uno completo, con $n = 3$.

Teorema (Ben-Or, 1983, citado en C2 §8)

Distinción de elementos — dados n reales, decidir si hay dos iguales — requiere $\Omega(n \log n)$ en árboles de decisión algebraicos.

Teorema

Par más cercano en 1D requiere $\Omega(n \log n)$ en árboles de decisión algebraicos.

Demostración. En el plano C2 §8 necesita el mapa $x_i \mapsto (x_i, 0)$ porque la entrada del problema son puntos de $\mathbb{R}^2$. En 1D los n reales **ya son** una instancia de par más cercano: la reducción es la identidad y cuesta **cero** pruebas de signo.

Sea A un árbol de profundidad $T(n)$ para el par más cercano. En cada hoja de A queda un par (i, j), $i \neq j$, que es el más cercano para toda entrada que llega ahí. Si algún valor está repetido, la distancia mínima es 0, así que el par (i, j) de la hoja cumple $x_i = x_j$; si todos son distintos, $x_i \neq x_j$. Basta entonces colgar de cada hoja **un nodo más**, que pruebe el signo del polinomio $x_i - x_j$ (grado 1): "$= 0$" responde *hay repetidos*, "$\neq 0$" responde *todos distintos*. El árbol resultante resuelve distinción de elementos con profundidad $T(n) + 1$. Si fuera $T(n) = o(n \log n)$, contradiría a Ben-Or. □

Verificado (5000 instancias con repetidos forzados): distancia 0 si y solo si hay un repetido.

```
reduccion (distancia 0 syss hay repetido): 5000 pruebas, fallos (deberia ser 0): 0
```

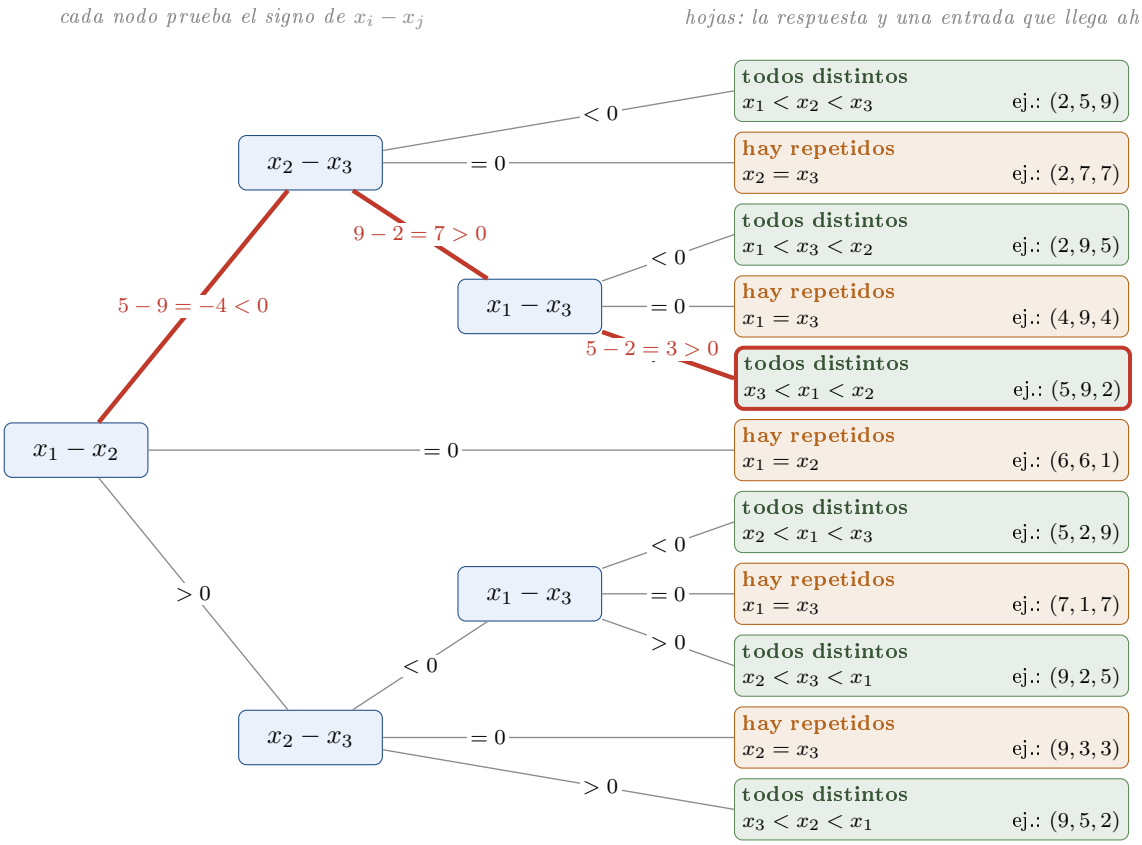


Figura 1: Un árbol de decisión completo para distinción de elementos con $n = 3$ (pruebas lineales, o sea algebraico de grado 1). Cada nodo prueba el signo de $x_i - x_j$ y ramifica en tres. Las hojas verdes responden *todos distintos* y las naranjas *hay repetidos*; cada una muestra un orden o igualdad y una entrada concreta que llega ahí. En rojo, el camino de la entrada $(5, 9, 2)$: $5 - 9 = -4 < 0$, luego $9 - 2 = 7 > 0$, luego $5 - 2 = 3 > 0$, y termina en la hoja $x_3 < x_1 < x_2$. Profundidad 3, 11 hojas, y exactamente $6 = 3!$ son verdes: una por cada orden posible. El árbol del par más cercano es el mismo tipo de objeto, con un par (i, j) en cada hoja.

1.1 Una demostración sin citar a Ben-Or (modelo de comparaciones)

En un modelo más restringido la cota admite una demostración elemental, que no cita a Ben-Or. Si todas las pruebas son **lineales** — en particular, comparar x_i con x_j, que es el modelo de C5 — la región de entradas que llega a una hoja es convexa, y basta contar hojas con la técnica de C5 §3 (es el ejercicio 5 de esa conferencia). Con pruebas polinomiales de grado arbitrario esa región puede tener muchas componentes conexas, y por eso Ben-Or necesita el teorema de Milnor y Thom para acotarlas (se vuelve sobre esto al final de la subsección).

Lema

En un árbol de decisión lineal correcto para distinción de elementos, dos entradas x, y de números **distintos** con **órdenes distintos** llegan a hojas distintas.

Demostración. La región de entradas que llegan a una hoja es una intersección de semiespacios abiertos ($\ell(x) < 0$, $\ell(x) > 0$) e hiperplanos ($\ell(x) = 0$): es **convexa**. Supongamos que x e y llegan a la misma hoja. Como sus órdenes difieren, hay índices i, j con $x_i < x_j$ e $y_i > y_j$. La función $t \mapsto \big((1-t)x_i + ty_i\big) - \big((1-t)x_j + ty_j\big)$ es continua, vale < 0 en $t = 0$ y > 0 en $t = 1$, así que se anula en algún $t^* \in (0, 1)$. El punto $z = (1 - t^*)x + t^*y$ está en la región (convexidad) y tiene $z_i = z_j$: es una entrada *con repetidos* que llega a una hoja a la que también llega x, cuya respuesta es "todos distintos". El árbol se equivoca con z. □

Hay $n!$ órdenes, luego al menos $n!$ hojas con respuesta "todos distintos" (en la figura 1 son las 6 hojas verdes). Un árbol ternario de profundidad h tiene a lo sumo 3^h hojas (C5 §3, lema 1, con tres ramas), así que

$$h \;\geq\; \log_3 n! \;\geq\; n \log_3 \frac{n}{e} \;=\; \Omega(n \log n),$$

usando $n! \geq (n/e)^n$, que es la cota fina del lema 3 de C5 §3 (la elemental, $\log n! \geq \frac{n}{2} \log \frac{n}{2}$, ya alcanza para $\Omega(n \log n)$). La diferencia con Ben-Or es solo qué hay que cuidar: con grado d la región de una hoja deja de ser convexa y puede tener muchas componentes conexas, y por eso hace falta Milnor–Thom para acotarlas. Lo que se cuenta es lo mismo en ambos casos: el conjunto "todos distintos" tiene $n!$ componentes conexas, una por orden.

Verificado sobre un algoritmo de comparaciones concreto (insercion binaria): las $n!$ entradas distintas terminan en $n!$ hojas distintas, y la profundidad queda por encima de la cota.

n	n!	hojas distintas	prof. maxima	ceil(log2 n!)	ceil(log3 n!)
3	6	6	3	3	2
4	24	24	5	5	3
5	120	120	8	7	5
6	720	720	11	10	6
7	5040	5040	14	13	8
8	40320	40320	17	16	10

(La tabla ilustra el fenómeno en un algoritmo; la demostración para *todos* es el lema.)

Conclusión. Ordenar y mirar consecutivos (C2 §3) cuesta $O(n \log n)$, así que es **óptimo** en el modelo de árboles de decisión algebraicos.

2 Qué algoritmo rompe la cota, y en qué modelo

El algoritmo que rompe la cota

El algoritmo **aleatorio incremental** de C2 §9 (Rabin 1976 / Kleinberg–Tardos §13.7): costo **esperado** $O(n)$, en una RAM con **parte entera** ($\lfloor \cdot \rfloor$) y **direccionamiento indirecto** (una tabla hash de costo $O(1)$ esperado) — dos operaciones que un árbol de decisión algebraico no tiene (C2 §10).

No hay contradicción con la sección 1: Ben-Or vale *solo* para árboles algebraicos. Tres precisiones:

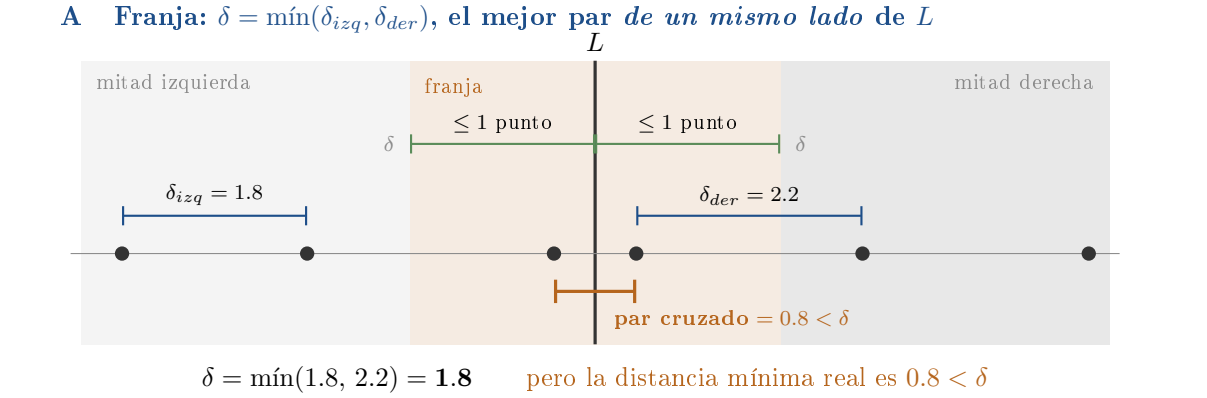
- **Qué operación hace qué.** $\lfloor x/\delta \rfloor$ no es un polinomio ni es continua, y el resultado se usa como *dirección* de la tabla. El teorema de Ben-Or no puede ver ninguna de las dos.
- **La reducción sigue valiendo en el modelo nuevo.** Lo que cambia es lo que transfiere: en la RAM con hash, distinción de elementos cuesta $O(n)$ esperado (una tabla hash sobre los valores, sin rejilla ninguna), así que la reducción solo da $\Omega(n)$, que es leer la entrada. En ese modelo los dos problemas son $\Theta(n)$. La parte entera es lo que permite pasar de "¿hay iguales?" a "¿cuál es el hueco mínimo?": la rejilla agrupa por cercanía, no por igualdad.
- **Hay dos fuentes de azar.** El barajado de los puntos (que analiza la sección 3.5) y la función hash (cuyo $O(1)$ esperado se asume, por ejemplo con hashing universal). Y la parte entera $\lfloor x/\delta \rfloor$ tiene que caber en una palabra (C1 §9): si δ es minúsculo respecto del rango de los puntos, el índice de celda tiene muchos bits y el $O(1)$ por operación deja de ser razonable.

3 Adaptación a una dimensión

3.1 Qué es δ aquí, y qué era en la franja

En C2 §5–6 (divide y vencerás) $\delta = \min(\delta_{izq}, \delta_{der})$ es el mejor par *del mismo lado* de la recta de corte: una cota superior de la distancia mínima real, que los pares cruzados pueden mejorar. En C2 §9 y en este ejercicio δ es la distancia mínima *exacta* entre todos los puntos ya insertados. La figura 2 lo muestra con los *mismos* puntos. Las dos veces la conclusión es "una celda tiene a lo sumo un punto", pero por razones distintas:

	δ **es...**	**una celda tiene** ≤ 1 **punto porque...**
Franja (C2 §6)	mínimo de los pares del mismo lado (cota superior de la distancia real)	cada celda cae en un solo lado de L y ahí todo par está a $\geq \delta$
Rejilla (C2 §9, aquí)	mínimo exacto de los puntos ya insertados	δ ya es la distancia mínima de *todos* los pares insertados



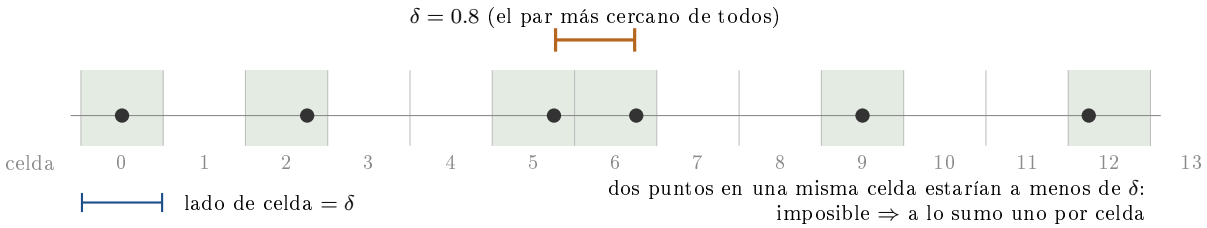


Figura 2: Los mismos seis puntos, dos significados de δ. **(A)** En la franja, δ es el mejor par *de un mismo lado* de L ($\delta_{izq} = 1.8$, $\delta_{der} = 2.2$, luego $\delta = 1.8$). No es la distancia mínima real: el par cruzado (naranja) está a $0.8 < \delta$, y encontrarlo es justo para lo que sirve la franja. Los dos intervalos de longitud δ junto a L tienen a lo sumo un punto porque los pares del mismo lado están a $\geq \delta$. **(B)** En la rejilla, δ es la distancia mínima *exacta* de todo lo insertado, o sea 0.8: el par naranja, que en (A) quedaba fuera de la definición de δ, ahora la define. Cada celda de lado δ (sombreada si está ocupada) tiene a lo sumo un punto.

3.2 El lado y la ventana, rehechos para 1D

C2 §9 usa celdas de lado $\delta/2$ y una ventana de 5×5: eso es una elección del plano, no una ley. En 1D la cuenta es esta. Las celdas son los intervalos semiabiertos $[k\ell, (k+1)\ell)$ con $\ell = \delta > 0$, y la celda de x es $\lfloor x/\ell \rfloor$.

Lema A (una celda, un punto)

Si $\delta > 0$ es la distancia mínima entre los puntos de un conjunto S, ninguna celda de lado δ contiene dos puntos de S.

Demostración. Dos puntos de $[k\delta, (k+1)\delta)$ difieren en *menos* de δ, y ningún par de S está a menos de δ. □

Lema B (ventana de 3 celdas)

Si $|p - q| < \delta$, las celdas de p y q (de lado δ) difieren en a lo sumo 1.

Demostración. Sean $a = p/\delta$, $b = q/\delta$, con $|a-b| < 1$. Entonces $\lfloor a \rfloor - \lfloor b \rfloor < a-(b-1) = a-b+1 < 2$, y como es entero, ≤ 1. Simétricamente ≥ -1. □

La misma cuenta en dimensión d con celdas de lado $\delta/\sqrt{d}$ (la diagonal es δ) da:

d	lado	radio de la ventana	celdas a revisar
1	δ	1	3
2	$\delta/\sqrt{2}$ (C2 usa $\delta/2$)	2	$5 \times 5 = 25$
3	$\delta/\sqrt{3}$	2	$5^3 = 125$

(El radio es $\lceil \sqrt{d} \rceil$: el mismo argumento del lema B con $|a - b| < \sqrt{d}$.) Así que las 25 celdas del plano no se "adaptan" a 1D copiando el 5: se rehace la cuenta y salen 3. Lo que **no** funciona es mezclar las dos elecciones. Con lado $\delta/2$ hacen falta 5 celdas, no 3; con lado δ una sola celda no alcanza.

```
delta=1, lado=0.5: p=0.49, q=1.48, |p-q|=0.99 < delta, celdas 0 y 2, difieren en 2
delta=1, lado=1.0: p=0.9, q=1.1, |p-q|=0.20 < delta, celdas 0 y 1, difieren en 1

vecino a distancia < delta que queda fuera de la ventana (20000 ensayos):
   lado = 0.5 * delta, ventana de 5 celdas: 0
   lado = 0.5 * delta, ventana de 3 celdas: 5020
   lado = 1.0 * delta, ventana de 3 celdas: 0
   lado = 1.0 * delta, ventana de 1 celdas: 9924

algoritmo completo con cada variante (2000 instancias de n=10..60, resultado distinto del
    correcto):
   lado = 1.0 * delta, ventana de 3 celdas: 0 errores
   lado = 0.5 * delta, ventana de 5 celdas: 0 errores
   lado = 0.5 * delta, ventana de 3 celdas: 241 errores
   lado = 1.0 * delta, ventana de 1 celdas: 608 errores

maximo de puntos en una celda (5000 conjuntos): lado=delta -> 1, lado=delta/2 -> 1
```

La variante "lado $\delta/2$ con 3 celdas" no es un detalle de borde: una cuarta parte de los vecinos reales (5020 de 20000) quedan fuera de la ventana, y el algoritmo completo contesta mal en el 12 % de las instancias (241 de 2000).

3.3 El algoritmo

El listado es, literalmente, el código que ejecuta `verif.py`:

```python
def par_cercano_incremental_1d(P, semilla=0, mezclar=True):
    """Rejilla de lado delta, ventana de 3 celdas. Devuelve (delta, reconstrucciones,
        trabajo)."""
    pts = list(P)
    if len(pts) < 2:
```

```
        raise ValueError("hacen falta al menos 2 puntos")
    if mezclar:
        random.Random(semilla).shuffle(pts)
    delta = abs(pts[0] - pts[1])
    if delta == 0:                              # repetido: la respuesta es 0
        return 0, 0, 0
    celda = lambda x, lado: math.floor(x / lado)
    rejilla = {celda(pts[0], delta): pts[0], celda(pts[1], delta): pts[1]}
    reconstrucciones = trabajo = 0
    for i in range(2, len(pts)):
        p = pts[i]
        c = celda(p, delta)
        antes = delta
        for dc in (-1, 0, 1):                   # ventana de 3 celdas
            q = rejilla.get(c + dc)
            if q is not None and abs(p - q) < delta:
                delta = abs(p - q)
        if delta == 0:                          # p repite un valor ya insertado
            return 0, reconstrucciones, trabajo
        if delta < antes:                       # delta bajo: reconstruir la rejilla
            rejilla = {celda(q, delta): q for q in pts[:i + 1]}
            reconstrucciones += 1
            trabajo += i + 1
        else:
            rejilla[c] = p                      # la celda estaba vacia (invariante)
    return delta, reconstrucciones, trabajo
```

Dos observaciones sobre el código. Primero, el caso $\delta = 0$ (un valor repetido) **corta** la ejecución: la respuesta ya es 0, y seguir calcularía una celda $\lfloor x/0 \rfloor$. Este caso no es una rareza: es exactamente la instancia que usa la reducción de la sección 1, y en la práctica es la tabla la que resuelve distinción de elementos. Segundo, `rejilla[c] = p` no sobrescribe nada, y hay que demostrarlo (siguiente apartado).

3.4 Correctitud

Invariante $I(i)$, para $2 \le i \le n$

Tras procesar los primeros i puntos de la permutación, $S_i = \{p_1, \ldots, p_i\}$: (a) δ es la distancia mínima de S_i, y es > 0; (b) `rejilla` contiene exactamente los puntos de S_i, cada uno en la entrada $\lfloor x/\delta \rfloor$.

Inicialización ($i = 2$). Si $p_1 = p_2$ se devuelve 0, que es correcto. Si no, $\delta = |p_1 - p_2| > 0$ es la distancia mínima de S_2, y por el lema A los dos puntos están en celdas distintas: (b) vale.

Mantenimiento ($I(i) \Rightarrow I(i{+}1)$). Sea $p = p_{i+1}$ y δ el valor antes del paso. Los pares de S_{i+1} que no contienen a p están a $\ge \delta$. Un punto $q \in S_i$ con $|p - q| < \delta$ está, por el lema B, en una de las 3 celdas de la ventana, y por (b) la tabla lo contiene. Luego tras el escaneo $\delta' = \min(\delta, \min_{q \in S_i} |p - q|)$ es la distancia mínima de S_{i+1}. Tres casos:

- $\delta' = 0$: p repite un valor. Se devuelve 0, correcto.
- $\delta' < \delta$: se reconstruye la tabla con el nuevo $\delta' > 0$ y el conjunto S_{i+1}. Por el lema A no hay dos puntos en una celda, así que ninguno se pierde, y (a), (b) valen.
- $\delta' = \delta$: ningún punto de la ventana está a menos de δ de p. Pero si la celda de p tuviera un punto q, ese q estaría en la ventana y a menos de δ de p. Luego **la celda de p está vacía**, insertar no sobrescribe nada, y (a), (b) valen.

Conclusión. Con $i = n$, δ es la distancia mínima de todos los puntos. **Terminación**: el `for` recorre $n - 2$ índices y cada reconstrucción es finita. □

Verificado, contra los oráculos de C2 (ordenar, y fuerza bruta). El segundo bloque usa enteros, donde la aritmética es exacta, y hay repetidos en el 37 % de las instancias, justo el caso que una prueba con reales uniformes nunca ejercita:

```
reales uniformes : 3000 instancias, fallos (deberia ser 0): 0
enteros con repetidos: 3000 instancias (1113 con respuesta 0), fallos (deberia ser 0): 0
```

3.5 Costo esperado

Cada paso hace a lo sumo 3 consultas y una inserción: $O(1)$ esperado. Una reconstrucción en un conjunto de s puntos cuesta $\Theta(s)$. Todo depende de cuántas haya y de cuándo.

Análisis hacia atrás (C2 §9), incluyendo empates. Fijemos el conjunto S_s y olvidemos el orden en que llegó; el último punto insertado es uniforme entre sus s elementos, porque el orden es una permutación uniforme. Si el paso s reconstruye, δ bajó. Entonces *todo* par más cercano de S_s contiene a p_s: si alguno no lo contuviera, ya estaría en S_{s-1} y δ no habría bajado. Fijado un par más cercano (a, b), $p_s \in \{a, b\}$, y eso son a lo sumo dos de los s candidatos:

$$\Pr[\text{el paso } s \text{ reconstruye}] \;\le\; \frac{2}{s}.$$

(C2 §9 supone el par único; aquí no hace falta.) Por linealidad de la esperanza,

$$\mathbb{E}[\text{trabajo de reconstrucción}] \;\le\; \sum_{s=3}^{n} \frac{2}{s} \cdot c\,s \;=\; 2c(n-2) \;=\; O(n),$$

y con los n pasos de $O(1)$ esperado cada uno, $\mathbb{E}[\text{costo}] = O(n)$. En el caso genérico (par más cercano único) las cuentas son exactas: el número esperado de reconstrucciones es $\sum_{s=3}^{n} 2/s = 2H_n - 3 \approx 2\ln n - 1.85$, y el trabajo esperado es $2(n-2)$.

Verificado: media sobre barajados distintos de un *mismo* conjunto. La columna $2\ln n$ de C2 sobrestima por la constante $3 - 2\gamma \approx 1.85$, y $2H_n - 3$ acierta; una sola corrida no sirve para distinguirlas, porque la desviación típica es de 3 a 5.

n	semillas	media	desv	2 ln n	2H_n-3	trabajo/n
1000	200	11.96	3.42	13.82	11.97	1.94
10000	200	16.37	3.81	18.42	16.58	1.98
100000	200	21.31	4.52	23.03	21.18	2.04
1000000	20	26.65	5.27	27.63	25.79	1.94

3.6 El peor caso, y qué hace improbable el barajado

Sea $x_k = 2^{-k}$, insertados en el orden $k = 1, 2, \ldots, n$. Los huecos entre consecutivos son $2^{-(k+1)}$, decrecientes, así que cada punto nuevo forma un par a distancia menor que todos los anteriores: **todos** los pasos reconstruyen, el trabajo es $\sum_s s = \Theta(n^2)$.

La garantía es para toda entrada. Esa misma entrada, barajada por el algoritmo, cuesta $O(n)$ en esperanza. Lo improbable no es una entrada: es un *orden de inserción*. La posición relativa de p_s entre los s primeros es uniforme e independiente de las demás, y en esta entrada el paso s reconstruye si y solo si p_s es uno de los dos menores de S_s. Luego

$$\Pr[\text{reconstruyen todos los pasos}] \;=\; \prod_{s=3}^{n} \frac{2}{s} \;=\; \frac{2^{n-1}}{n!},$$

que decrece más rápido que cualquier exponencial. Se verificó enumerando las $n!$ permutaciones (no muestreando):

```
sin barajar (orden k=1..n): 798 reconstrucciones en 798 pasos, trabajo = 320,397 (n^2/2 =
    320,000)
barajando (200 semillas) : reconstrucciones media 11.3, trabajo medio 1,627, maximo 4,935
orden de insercion en el que TODOS los pasos reconstruyen (enumerando las n! permutaciones):
   n=5: 16 de 120 permutaciones; 2^(n-1) = 16; probabilidad 2^(n-1)/n! = 0.1333
   n=6: 32 de 720 permutaciones; 2^(n-1) = 32; probabilidad 2^(n-1)/n! = 0.0444
   n=7: 64 de 5040 permutaciones; 2^(n-1) = 64; probabilidad 2^(n-1)/n! = 0.0127
   n=8: 128 de 40320 permutaciones; 2^(n-1) = 128; probabilidad 2^(n-1)/n! = 0.0032
```

El peor caso *sobre las monedas del algoritmo* sigue siendo $\Theta(n^2)$, pero con probabilidad $2^{n-1}/n!$. Comparar con C4: un costo *amortizado* es una garantía de peor caso sobre secuencias, sin probabilidad; un costo *esperado* es un promedio sobre el azar del algoritmo, válido para cada entrada. Ninguno es un "caso promedio" sobre una distribución de entradas.

3.7 Mediciones honestas

Contar operaciones de tabla sí es independiente de la máquina: 3 consultas y 1 inserción por punto, más ≈ 2 reinserciones por las reconstrucciones (columna `trabajo/n` de arriba): unas 6 operaciones por punto, con n cualquiera. Ordenar hace del orden de $\log_2 n$ comparaciones por punto. Pero el *tiempo* cuenta otra historia. La tabla es una medición de referencia; sus tiempos concretos dependen de la máquina y de la carga:

```
      n   ordenar  rejilla ord us/pto rej us/pto log2 n rej/ord
  10000    0.002s    0.008s      0.171      0.760   13.3     4.4
 100000    0.022s    0.124s      0.218      1.241   16.6     5.7
1000000    0.386s    1.837s      0.386      1.837   19.9     4.8
```

En Python la rejilla es de 4 a 6 veces **más lenta** que ordenar y mirar consecutivos (varía entre corridas; en otras salió hasta 8). Hay dos razones y ninguna contradice el análisis: `sorted` está escrito en C, mientras que el bucle de la rejilla es intérprete puro, y a $n = 10^6$ un factor $\log_2 n$ es solo ≈ 20. Con estos tamaños el tiempo *por punto* crece en ambos (sube a $\approx 2.4\times$ en la rejilla y a $\approx 2.3\times$ al ordenar), así que **no se alcanza a ver** la diferencia entre $O(n)$ y $O(n \log n)$; lo más probable es que domine la jerarquía de memoria (cada consulta a una tabla de 10^6 entradas es un acceso aleatorio), que el modelo RAM de C1, donde todo acceso cuesta 1, no capta. Conclusión que sí es defendible: la rejilla rompe la cota *en el modelo*, no gana por ser lineal en las máquinas y tamaños de aquí.

3.8 Qué le pasa al 7 de C2 §6 en una dimensión

Es la pieza que une este ejercicio con la franja. En 1D, con $\delta = \min(\delta_{izq}, \delta_{der})$ (mejor par del mismo lado, como en la tabla de arriba), los puntos de la franja $(L - \delta, L + \delta)$ son los de la izquierda en $(L - \delta, L]$ y los de la derecha en $[L, L + \delta)$. Cada intervalo tiene longitud δ, así que dos puntos del mismo lado dentro de él estarían a menos de δ entre sí, contra la definición de δ_{izq} y δ_{der}. Luego **cada lado aporta a lo sumo un punto** y la franja tiene a lo sumo dos: hay que comparar *un* par. El 7 del plano (ocho cuadrados de lado $\delta/2$) se vuelve un 1, y el divide y vencerás colapsa en ordenar y mirar consecutivos (C2 §3).

```
3000 instancias: fallos 0, maximo de puntos de la franja a la izquierda de L: 1, a la derecha: 1
```

Errores que hay que vigilar

- **Copiar las constantes del plano.** Ni el lado $\delta/2$ ni la ventana de 25 celdas se llevan a 1D tal cual. Con lado $\delta/2$ hacen falta 5 celdas ($p = 0.49$, $q = 1.48$, $\delta = 1$ caen a dos celdas de distancia), y con 3 celdas el algoritmo falla en 241 de 2000 instancias; con lado δ bastan 3. Hay que rehacer la cuenta de cobertura, no "recortar" una ventana mirando los bordes.
- **Pensar que $\Omega(n \log n)$ contradice el $O(n)$ esperado.** No hay una cota mínima para "par más cercano en 1D": hay una por modelo. La de la sección 1 es de árboles algebraicos; el algoritmo de la sección 3 usa parte entera y direccionamiento indirecto.
- **Describir mal la garantía esperada.** No es un promedio sobre las entradas posibles: vale *para toda entrada*, con la esperanza tomada sobre el barajado que hace el propio algoritmo. El peor caso sobre esas monedas sigue siendo $\Theta(n^2)$, con probabilidad $2^{n-1}/n!$ en la entrada de la sección 3.6.
- **Olvidar $\delta = 0$.** Un valor repetido da lado 0 y una división por cero — y los repetidos son justo la instancia de la reducción. Hay que cortar con la respuesta 0 en cuanto aparece, y probar con entradas que los tengan (los reales uniformes casi nunca los generan).
- **En la reducción:** (i) la hoja de un árbol devuelve *índices*, no el número δ; hace falta el nodo extra que prueba $x_i - x_j$; (ii) en 1D la reducción cuesta 0, no $O(n)$.
- **Concluir "$O(n)$ es más rápido".** Medido arriba: 4 a 6 veces más lento que ordenar, en Python y hasta 10^6. La notación asintótica habla de cómo crece, no de quién gana a un tamaño dado.
- **Una sola corrida como evidencia.** El número de reconstrucciones tiene desviación típica de 3 a 5, así que una sola corrida (por ejemplo 24 reconstrucciones con $n = 10^5$, donde $2 \ln n = 23.0$ y $2H_n - 3 = 21.2$) no distingue entre las dos fórmulas. Se promedia sobre barajados y se compara con $2H_n - 3$.
- **Aritmética inexacta.** La demostración supone aritmética exacta. Con reales de coma flotante, $\delta = |p - q|$ y $\lfloor x/\delta \rfloor$ se redondean, y dos puntos casi a distancia δ podrían caer en la misma celda (no se observó en las pruebas, pero la demostración no lo cubre). Con enteros y división entera exacta, el argumento se cumple al pie de la letra: por eso la prueba de la sección 3.4 incluye enteros.